spec-loop: A Fact-Gated, Tool-Agnostic Methodology for Specification-Driven Development with LLM Coding Agents

Andrés Massello
Córdoba, Argentina
github.com/andresmassello

July 2026

Abstract

LLM coding agents fail in two well-documented ways: when instructions are underspecified they guess instead of stopping, and when a verification signal is visible they optimize the signal rather than the requested outcome. We present **spec-loop**, a methodology for specification-driven development with LLM coding agents organized around a single discipline: *gates that read facts block; gates that guess over prose advise*. The methodology contributes (i) two specification fronts — a greenfield front where the agent *proposes* the system shape under one-question-at-a-time human approval, and a brownfield front where the agent may only *extract* verifiable facts, never author a specification of existing behavior; (ii) a deterministic evidence ledger with a two-tier record contract in which measured artifacts always override agent self-reports; (iii) a family of executable fact gates (golden non-authorship, gate-integrity, simplicity budgets, per-criterion measured acceptance, regression capture, a derived — never self-declared — workflow state machine) and a structured-guess layer (versioned qualitative rubrics with mandatory evidence) that advises by default and gates only by explicit human declaration; and (iv) a strict placement of the human at three fixed points: approving the shape, approving the golden baseline, and merging. A reference implementation (seven agent skills and a dependency-free Python engine with 23 subcommands and adapters for nine language stacks) is described, together with its design validation: two adversarial audits (231 agent evaluations) that initially found the central principle inverted in code and drove its re-wiring; a systematic cross-examination against classical software-engineering doctrine that produced ten shipped improvements; and sustained self-application, in which fresh-context reviews caught real defects in 13 of 14 releases before merge. We report design validation and self-application evidence only; controlled empirical evaluation remains future work.

1 Introduction

Two recent empirical results frame the problem this methodology addresses. First, when task instructions are underspecified, production coding agents do not fail closed: they act on unstated assumptions, and a majority of runs violate at least one action boundary [1]. Second, when a verification oracle is visible to the agent, near-perfect scores can coexist with an undelivered artifact: the agent *builds to the test*, optimizing the check rather than the request, and does not on its own validate what it ships as a user would [2]. Both failures are dispositional rather than capability failures, and both worsen as agents are given more autonomy.

Existing agent development frameworks converge on persistent artifacts and human review as mitigations [3], but they largely encode their process as *prose instructions that the agent may ignore*, and their verification signals rarely distinguish between what is mechanically verifiable and what is a model's opinion. spec-loop starts from that distinction and makes it the organizing principle of the whole process.

The methodology is built on one discipline and three architectural commitments:

Facts block; guesses advise.

Every check in the process is classified as *computational* (it reads an artifact — a byte comparison, a parsed test report, a diff structure) or *inferential* (it judges prose or code quality). Computational checks are allowed to block progress mechanically. Inferential checks may only advise, unless a human explicitly declares them gating. A natural-language heuristic that blocks produces false positives, gets disabled, and dies; keeping guesses advisory is what keeps them alive.

Measured beats narrated.

The evidence ledger records two tiers: measured records parsed from real artifacts (test reports, coverage files, linter output) and self-reported agent counts. A measured red always overrides a narrated green. Acceptance criteria close per criterion only when a green, name-tagged test case exists in the ingested reports — a checked checkbox without a test is recorded as *narrated-only* and does not close.

The human at three fixed points.

The human approves the *shape* (specification front), approves the *golden baseline* (the one artifact the agent is mechanically forbidden from authoring), and *merges*. The agent proposes, measures, and stops.

Tool-agnosticism by contract.

The engine is dependency-free Python that never runs an LLM: it validates structures and ingests reports. Wherever a model's judgment participates (e.g. rubric grading), the interface is a JSON contract; who produced the JSON — one vendor's agent, another's, or a human — is irrelevant to the ledger.

This paper describes the methodology (§3), its evidence engine (§4), the reference implementation (§5), and the design-validation evidence accumulated so far (§6), followed by limitations (§7) and future work (§8).

2 Related Work

Underspecification. Ji et al. [1] show that underspecified operational instructions cause agents to guess rather than stop, violating action boundaries in 55.8–67.8% of runs. spec-loop's greenfield front is a direct response: the specification is produced through a structured interrogation in which the agent proposes and the human decides, and inviolable constraints are captured in a constitution file whose breach is a first-class blocker, never a trade-off.

Building to the test. Ma et al. [2] demonstrate that agents deliver what is checked, not what is requested, and name the missing disposition *validation self-awareness*. spec-loop treats this as its central adversary: acceptance closes per criterion on measured test evidence; findings cannot be closed without new test material (*find bugs once*); qualitative verdicts require file-and-line evidence or they do not count; and the workflow state that authorizes a pull request is derived from ledger facts, never self-declared by the agent.

Process taxonomies. de Macedo [3] surveys agent development frameworks along six dimensions (specification, context, roles, execution, validation, portability) and identifies recurring risks: specification drift, artifact fragility, and platform dependence. spec-loop addresses each explicitly: drift through a rebuild test that scores whether the specification package alone can regenerate the system; fragility through an integrity-checksummed ledger; and platform dependence through the contract architecture and a dependency-free engine.

Practitioner doctrine. The methodology's check classification adapts the computational-versus-inferential distinction discussed in the practitioner literature on generative AI in software delivery [6]. Architecture decision records follow Nygard [5], extended with machine-checkable implementation plans. Golden-master capture follows approval-testing practice [7], with one addition central to the method: the agent may author the capture harness but is mechanically forbidden — by a pre-tool-use hook, not by instruction — from writing an

approved fixture. The methodology was also systematically cross-examined against classical doctrine [4]; §6 reports the ten resulting improvements. The design stance throughout is an application of Goodhart's law [8] to agent verification: any signal the agent can see, it will eventually optimize.

3 The Methodology

3.1 Two specification fronts

Greenfield (discovery). The human brings an idea; the agent brings the *shape*. The front proceeds one question at a time, each question carrying the agent's recommended answer, walking a fixed agenda: purpose, domain model, operation surface, architecture options with trade-offs, dirty cases and failure behavior, inviolable constraints, out-of-scope, acceptance criteria with stable traceable identifiers, a declared quality bar, and residual risks. High-uncertainty risks trigger a time-boxed *spike* whose only legitimate output is a decision record with lessons — spike branches are mechanically refused at the pull-request gate. The front emits a versioned specification package: context glossary, constitution, domain model, specification, decision records with implementation plans and verification checklists, acceptance file, risks, and handoff.

Brownfield (reverse discovery). For an existing system, the direction of trust inverts: observable behavior is ground truth, and the agent may produce *only facts* — a system map from static analysis and a golden suite captured mechanically at the boundaries. The agent never authors a specification of what the old system does or why: a specification written by the agent about legacy code encodes the same partial reading of the code that loses behavior silently, which is precisely the blind spot the golden exists to counter. The human writes the migration specification by reading the facts.

3.2 The development loop

The loop consumes the specification package and proceeds through phases, each guarded: plan (constitution first; no acceptance criteria, no build); a coverage gate that triggers boundary characterization tests when the safety net is insufficient; build under decision-record discipline (the agent consults records before touching governed areas and must propose — never silently take — new architectural decisions); a simplicity gate scoring the diff against declared budgets, with test code excluded so that writing tests is never penalized; a severity-gated review loop that *converges instead of chasing zero* (findings below the declared gate are deferred, not fixed forever); an integration pass across repositories; late test-writing against stabilized code; an optional rebuild test measuring specification completeness; and a pull request opened only when the derived workflow state says the facts allow it. The loop stops at merge: merging is human.

An escalation contract enumerates the situations in which the agent must stop and ask — iteration cap, oscillating findings, a previously-passing test now failing non-trivially, contradictory tool directives, decisions of architectural weight, and any constitution breach. Escalations and their closures are recorded events; an open escalation caps the readiness score until a human resolves it, and resolving a blocker requires an *escape analysis*: which gate or test should have caught this, and what was done about it.

4 The Evidence Engine

The engine is a single dependency-free Python file. It never runs an LLM and never runs the tools it scores: it *ingests* their reports (JUnit-family XML, Cobertura and lcov coverage, nine linter formats across nine language stacks) and validates structures. Every verdict is persisted in a deterministic ledger with an integrity checksum; external mutation or a truncated write blocks loading with a recovery message.

Fact gates (block). A byte-comparison golden gate (with declared volatile fields masked *visibly* — masking is reported, never silent); a gate-integrity check that blocks diffs that weaken the measuring apparatus (deleted or disabled tests across all nine stack conventions, lowered thresholds, added secrets); simplicity budgets over the diff; structural specification linting (missing sections, zero traceable acceptance criteria, duplicate identifiers); and a derived state machine: the workflow state (plan, build, qa, escalated, pr-ready) is *computed* from ledger facts. A self-declared state machine would be narrated state — the exact category of signal the methodology distrusts — so there is nothing to declare and no illegal transition to police.

Structured guesses (advise). Prose heuristics over specifications; regression capture (closing findings without adding a single non-blank test line is flagged as *narrated closure*); plateau and stop-signal advisories over the finding history; and a rubric layer: a versioned file of weighted qualitative criteria with anchor examples, negative criteria, and a threshold, graded by any runner against a JSON contract with evidence-or-nothing semantics — a verdict that affects the score without a file-and-line citation does not count. All of these advise by default and gate only when the human declares it in configuration; the declaration distinction is surfaced everywhere as *requirement (declared) versus default (kit opinion)*.

Readiness. A weighted 0–100 score reports the state of the result, never effort. The dominant dimension is measured acceptance — criteria closed by green, name-tagged tests — with checkbox completion demoted to a narrative dimension. Hard caps override the average (red tests, open blockers, unresolved escalations), and every biting threshold states its provenance. The score reports; it does not gate — the gates are the facts above.

5 Reference Implementation

The reference implementation packages the methodology as seven agent skills (the two fronts, a precision interview for known features, the loop orchestrator, golden capture, a two-audience system-documentation generator, and the rubric grader adapter) plus the engine (23 subcommands). Three properties matter for portability. First, the skills are markdown instruction files readable by any instruction-following agent; the engine is invoked identically from any of them. Second, every LLM-judgment interface is a contract: the rubric grader ships as a neutral prompt usable by any vendor's agent, a raw API call, or a human filling the JSON by hand — the implementation's test suite exercises exactly that path, with no LLM in the loop. Third, the one mechanical prohibition (the agent must not write approved golden fixtures) is enforced by a pre-tool-use hook at the runtime boundary, not by instruction — during development of the implementation itself the hook blocked one of its author's own commands, which is the property working as designed.

The implementation includes an installation-diagnosis subcommand in the spirit of `flutter doctor` (interpreter, toolchains per stack, hook presence and registration, skill integrity, per-project configuration), a smoke suite of 140 executable checks over a synthetic ledger, and three installation modes (per-project, per-user, and as a packaged plugin for one agent runtime — packaging, not dependency).

6 Design Validation and Self-Application

No controlled external evaluation has been conducted yet; we report the design-validation evidence honestly and completely.

Adversarial audits. Before stabilization, the methodology and its implementation were subjected to two adversarial audits totaling 231 agent evaluations: a seven-lens audit (54 agents; findings attacked by skeptics defaulting to *refuted*) confirmed 33 weaknesses and refuted 13, with the central finding that the core principle was *inverted in the code* — fact gates existed but were not wired to block. A second audit verified 171 claims across documentation, code, and design intent. The response re-wired every fact gate into the engine and re-

passed all documentation against the implemented reality (130 truthfulness edits), converting the principle from slogan to enforced property.

Doctrine cross-examination. The methodology was systematically read against classical software-engineering doctrine [4], producing 8 validations, 12 tensions with recorded resolutions, and 10 actionable improvements — all ten implemented, each as a separate release with its own regression checks. In the two deepest tensions the methodology deliberately departed from the letter of the doctrine to defend its own principle: readiness caps continue to bite by default (their existence is a definition; only the numbers are opinion, and they now state their provenance), and the workflow state machine is derived rather than declared.

Self-application. The implementation is developed under its own process. Across fifteen releases, each engine change carried smoke checks in the same commit, a fresh-context review before merge, and a five-way version consistency check enforced by the suite. Fresh-context reviews found real defects prior to merge in 13 of 14 reviewed releases, including: test-file classifier gaps that made a documented guarantee an overclaim; a blank-line loophole in regression-capture evidence; a review-suite check that passed for the wrong reason (its precondition never held); and silent last-wins semantics on duplicated grader verdicts — a grader gaming vector. We take the consistency of this defect stream as evidence for the methodology's core premise: independent, fresh-context checking against recorded criteria catches what the authoring context cannot see.

7 Limitations and Threats to Validity

No controlled evaluation. All evidence is design validation and self-application by the methodology's author; effect sizes on team productivity, defect escape rates, or comparison against the frameworks surveyed in [3] are unmeasured. **Single-operator bias.** Self-application shares authorship context; although reviews run in fresh contexts, they share the model family and the author's configuration. **LLM non-determinism.** The rubric layer's grades vary across runs; the methodology bounds the blast radius (advisory by default, evidence-or-nothing, human-declared gating) but does not eliminate variance. **Scope.** The loop targets a single operator driving one non-trivial change; multi-operator concurrency is out of scope, and the ledger is single-writer by design. **Heuristic residue.** Some gates carry documented approximations (path-based test classification, stale-report windows); each is disclosed at the point of use, but disclosure is not elimination.

8 Future Work

Planned next steps are: read-only dry runs of all nine stack adapters against real repositories; a multi-project dogfooding study with operators other than the author; a controlled comparison against the frameworks in [3] along their six taxonomy dimensions, with particular attention to the validation and portability axes; and an evaluation of the rubric layer's inter-run variance under anchored versus unanchored criteria, connecting to the validation self-awareness agenda of [2].

9 Conclusion

spec-loop's contribution is not a new agent architecture but a discipline for placing trust: facts block, guesses advise, the measured overrides the narrated, and the human decides at exactly three points. The methodology's own history — a central principle found inverted by adversarial audit, then wired into an engine that now blocks its own author when he crosses the line — suggests that for LLM-assisted development, the difference between a methodology and a slogan is whether the checks are executable. The reference implementation will be released as open source (MIT).

References

[1] Z. Ji, Z. Zhang, C. Xu, Z. Li, Y. Gao, S. Wang, and S.-C. Cheung, "Coding Agents Are Guessing: Measuring Action-Boundary Violations in Underspecified DevOps Instructions," arXiv:2607.02294 [cs.SE], 2026.

[2] Y. Ma, B. Kereopa-Yorke, and B. Schultz, "Building to the Test: Coding Agents Deliver What You Check, Not What You Requested," arXiv:2606.28430 [cs.SE], 2026.

[3] S. Oliveira de Macedo, "From Prompt to Process: a Process Taxonomy and Comparative Assessment of Frameworks Supporting AI Software Development Agents," arXiv:2606.04967 [cs.SE], 2026.

[4] D. Thomas and A. Hunt, *The Pragmatic Programmer: Your Journey to Mastery*, 20th Anniversary Edition. Addison-Wesley, 2019.

[5] M. Nygard, "Documenting Architecture Decisions," blog post, 2011. cognitect.com/blog/2011/11/15/documenting-architecture-decisions

[6] M. Fowler et al., "Exploring Generative AI," memo series, martinfowler.com, 2023–2026. martinfowler.com/articles/exploring-gen-ai.html

[7] L. Falco et al., "ApprovalTests," approvaltests.com.

[8] M. Strathern, "'Improving ratings': audit in the British University system," *European Review*, vol. 5, no. 3, pp. 305–321, 1997.